BACKEND DEVELOPER EXERCISE 2

Images CDN

A distributed, on-premise cluster that stores images and serves them over HTTP. The design, and a working prototype that demonstrates it.

Content-addressed storage | NATS JetStream replication log | Fetch on miss + read repair | Log replay + anti-entropy

Federico Pagnossin · September 2026

THE PROBLEM

Three requirements, one answer each

R1 · Synchronization

Nodes stay in permanent sync; a new image (POST) reaches every node as quickly as possible.

Upload event on a durable log that every node consumes; each node pulls the bytes from a node that has them, in parallel. The image is on 2 nodes before the `201`.

R2 · Read before sync

A node must return an image even if it has not been synchronized on that node yet.

Fetch on miss: the node asks the image's owners (which every node can compute), streams it to the client and keeps a copy (read repair).

R3 · Recovery

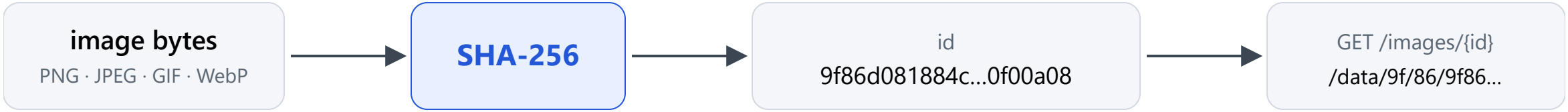
A node that goes down must resynchronize as quickly as possible at the next startup.

Replay of the log from the node's last acknowledged event: exactly what it missed. **Anti-entropy** as fallback. The node serves traffic while it catches up.

Constraint given: images are never deleted. The design turns this into its main simplification (next slide).

KEY IDEA

The id of an image is the SHA-256 of its bytes



Immutable, content-addressed and never deleted: the data of every node is a **set that only grows**.

No conflicts

Two nodes can never hold different versions of an id. Merging two nodes is a set union.

Idempotent

Same upload twice, same event twice, same file twice: no effect. Every mechanism can simply retry.

Integrity

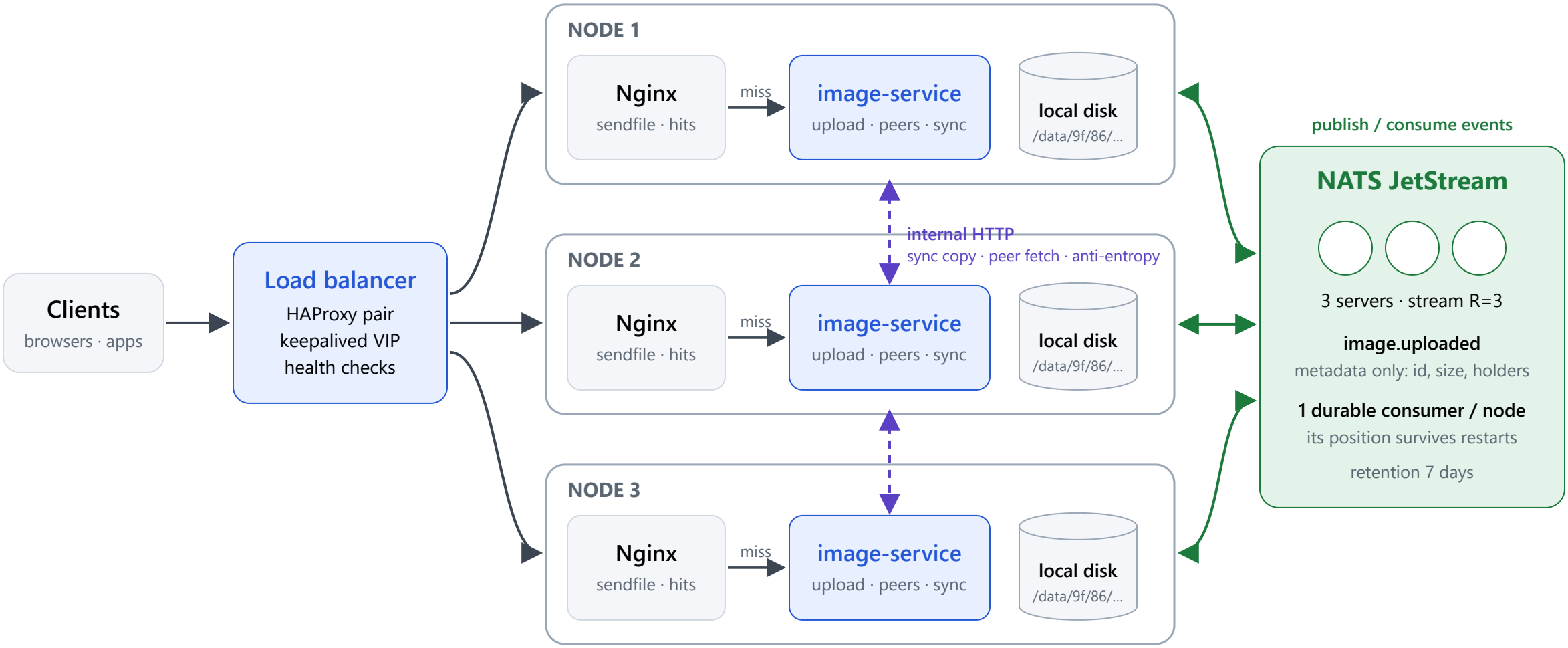
Every node verifies the hash of every file it receives. Corruption cannot spread.

Cache forever

`Cache-Control: immutable`, one year, `ETag = id`. Browsers and CDNs never revalidate.

ARCHITECTURE

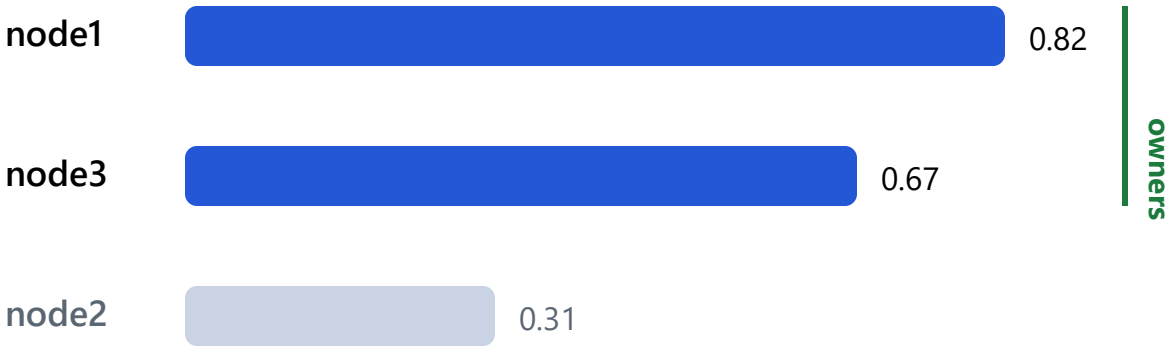
Identical nodes, a durable log, no leader



OWNERS

Every node knows where an image must be, without asking anyone

Rendezvous hashing: for image `9f86…`, each node scores `hash(node + id)`. The 2 highest are its **owners**.



Different images get different owners: load spreads evenly.

Used twice

POST copies the image **synchronously** to the first reachable owner. A GET miss asks the owners **first**, then every other node in parallel.

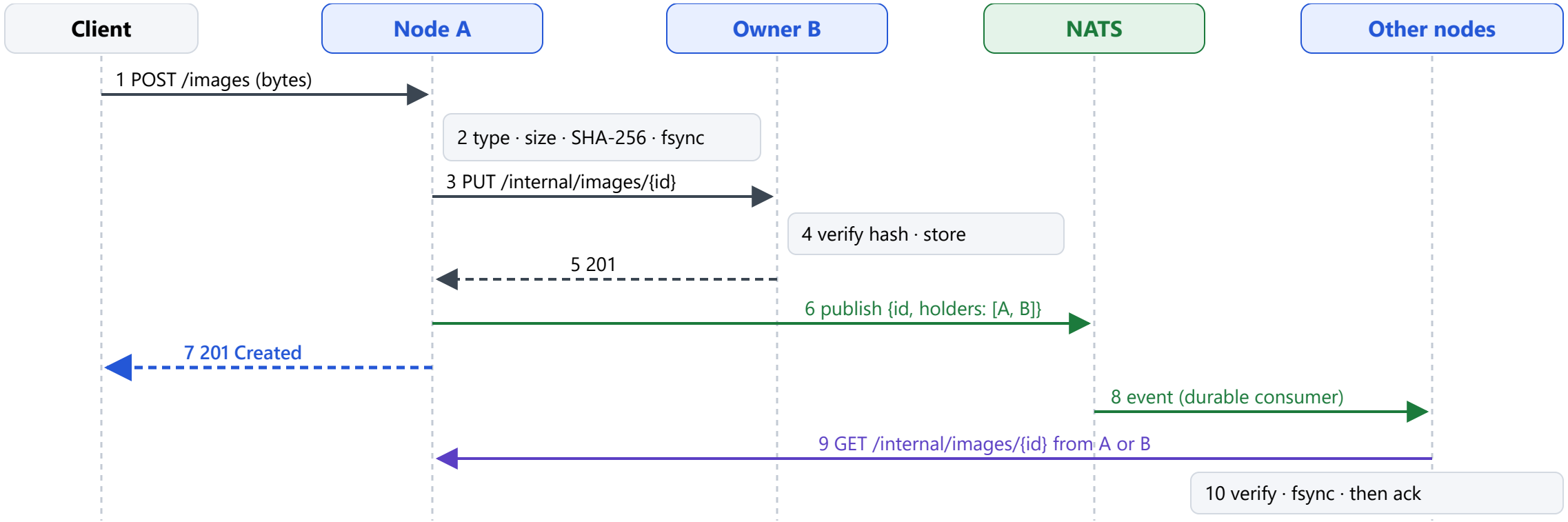
No coordination

No metadata service, no lookup table: a pure function of the id and the node list.

Stable

Adding or removing a node only moves the images that node owned. An owner down at upload time: the next node in the ranking takes the copy.

Durable on two nodes before answering, everywhere within a second



Durability

Write quorum 2: losing one disk right after the `201` loses nothing.

At-least-once

Ack only after fsync; redelivery is harmless (idempotent); publish deduplicated by id.

Log unavailable

Upload still accepted: the event waits in a persistent outbox.

Fetch on miss, from the owners first



Read-your-writes, any node

After a `201` an owner has the image, and every node can compute who the owners are.

Single flight

100 concurrent misses for one id: one peer fetch, shared.

Bulkhead

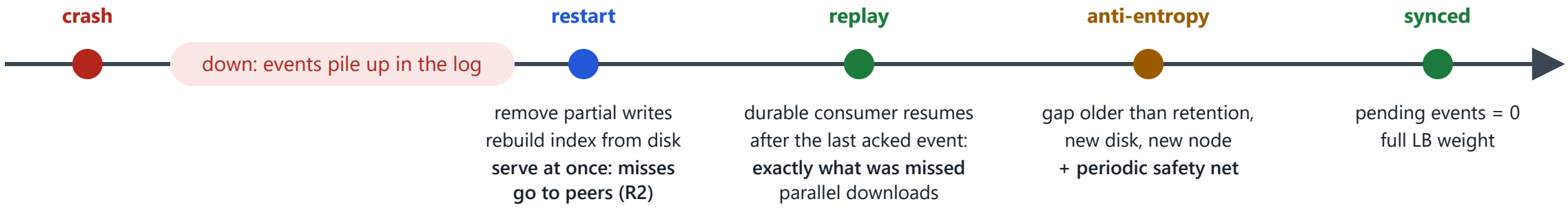
At most 32 fetches at once per node; beyond: fast `503 + Retry-After`.

Circuit breaker

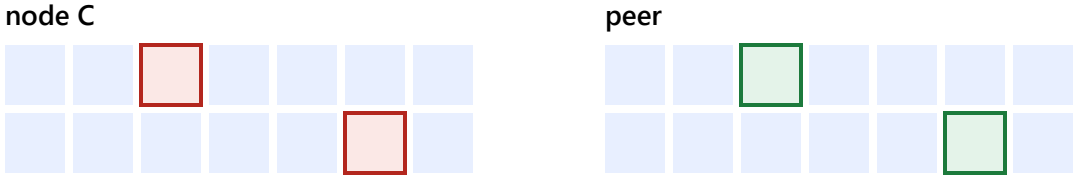
A dead or slow peer is skipped for a few seconds instead of slowing every request.

R3 · A NODE RESTARTS

Useful immediately, synchronized in seconds



Anti-entropy without scanning everything



4096 buckets (first 3 hex chars of the id), one digest each:
count + XOR of the ids, updated in O(1) at every insert.
Compare 4096 digests, list and download only the differing buckets.

Why the log is the fast path

The server keeps each node's position: no full scan, no diff, just the missed events.

Why the node serves while catching up

Fetch on miss covers the gaps, so recovery never blocks availability.

WHAT CAN GO WRONG

Failure scenarios

SCENARIO	BEHAVIOUR
A node dies	The load balancer removes it. Every image is on the other nodes; new uploads use the next owner in the ranking.
Uploading node dies right after 201	The owner has the synchronous copy; the event is in the log, or anti-entropy finds the image.
NATS unavailable	Uploads accepted through the persistent outbox, GETs work, replication delayed and then caught up.
Slow peer	2 s timeout and circuit breaker, then the next holder is tried.
Network partition	Both sides keep accepting uploads. Content-addressed ids cannot conflict: after healing, log and anti-entropy merge the sets.
Corrupted file	Hash verified on every transfer (and by a periodic scrub): the file is fetched again from a peer.
Disk almost full	Writes refused with `507` before the disk is full, and an alert; reads keep working.
Traffic burst on a cold node	Single flight + bulkhead: bounded peer fetches, fast `503` beyond, no cascade.

DECISIONS

Trade-offs, chosen deliberately

Eventual consistency (AP)

The requirement says "as quickly as possible", not "atomically on every node". Waiting for all nodes would make uploads slow and block them when one node is down. Guaranteed instead: durable on 2 nodes before `201`, readable from every node at once, everywhere in seconds.

Durable log, not direct push

Pushing to peers loses whatever a node missed while down. A replayable log with a per-node position is what makes recovery (R3) simple and exact.

Metadata on the log, bytes over HTTP

Images can weigh MB; the log stays small and fast, and bytes travel only between nodes.

NATS JetStream, Kafka as alternative

Same guarantees (durable consumer, ack, redelivery, dedup). JetStream is one binary, simple on premise. Kafka is fine if the company already runs it.

Full replication: the limit to know

Every node stores every image, as required: reads scale with nodes, capacity equals one node's disk. Beyond that, the same rendezvous hashing moves the cluster to partial replication (R copies per image) with no API change.

BUILT FOR HIGH TRAFFIC

Every path that could take a node down is bounded

Memory

Uploads read in chunks, cut at 10 MB even without Content-Length.

Concurrency

Bulkhead on peer fetches, bounded connection pools, bounded replication batches.

Latency

Timeout on every call, circuit breaker per peer, retries with exponential backoff + jitter.

Duplicates

Single flight per id; idempotent uploads make every retry and failover safe.

Crash safety

tmp + fsync + atomic rename + directory fsync; ack only after fsync.

Disk

`507` while 100 MB are still free: outbox, logs and renames keep working.

Scale of sync

O(1) digest updates; anti-entropy costs O(4096), not O(images).

Caching

Immutable one-year caching, `ETag`, `304`: most traffic never reaches a node.

Watch: replication lag per node (the key metric for R1 and R3), miss rate, peer-fetch latency, open breakers, 503/507 rates, disk usage.

PROOF

A working prototype of this design

Online (free)

Render, one container: NATS + 3 isolated nodes + gateway with a live dashboard. Crash and restart nodes from the page.

Offline

`docker compose up` : 3 node containers with their own volumes, NATS, gateway. Real `docker compose kill node3` .

Quality

60 tests (3-node in-process cluster, real NATS), `mypy --strict` , coverage gate, CI runs the three scenarios end to end.

DESIGN	CODE
Content-addressed store, digests	`storage.py`
Owners	`rendezvous.py`
POST, GET, replication, anti-entropy	`node.py`
Log, durable consumer	`events.py`
Breaker, single flight, bulkhead	`resilience.py`
Peer calls, integrity	`peers.py`
HTTP API, limits, 503/507	`node_app.py`

Simplified in the prototype: FastAPI instead of Nginx for hits, a single NATS server, gateway instead of HAProxy.

IN ONE SENTENCE

Content addressing removes conflicts; synchronous owners and fetch on miss make every node answer at once; a replayable log makes recovery exact, and anti-entropy makes it certain.

R1

Event on a durable log, pull from holders, 2 copies before 201.

R2

Owners computed locally, fetch on miss, read repair.

R3

Resume after the last ack, anti-entropy on 4096 digests.